VS Ingestion Architecture

Purpose

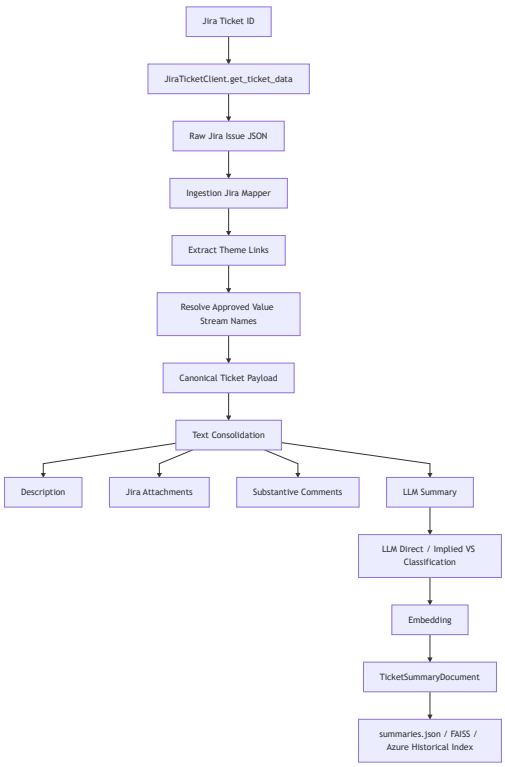
The ingestion pipeline converts a Jira ticket into a clean, LLM-generated, embedding-ready historical record for value-stream RAG.

The output of ingestion becomes the historical memory used later by RAG to identify direct and implied value streams for new idea cards.

Core principle:

```
Only index useful, verified historical tickets.
Do not index weak fallback summaries.
Do not invent value-stream labels.
```

High-Level Flow



Package Ownership

Current standalone ingestion package:

```
src/vs_app/ingestion/
```

Recommended mental model:

```
vs_app.ingestion
  = Jira ticket → historical RAG summary document

vs_app.integrations
  = external clients and generic file parsing

vs_app.modules.rag
  = retrieval, candidate merge, LLM adjudication
```

Important files

```
jobs/ingest_tickets.py
src/vs_app/ingestion/summary/pipeline.py
src/vs_app/ingestion/summary/text_consolidator.py
src/vs_app/ingestion/summary/llm_summary_extractor.py
src/vs_app/ingestion/summary/mapper.py
src/vs_app/ingestion/jira/mapper.py
src/vs_app/ingestion/jira/value_stream_labels/theme_extraction.py
src/vs_app/ingestion/jira/value_stream_labels/value_stream_mapping.py
src/vs_app/modules/tickets/documents.py
```

Entry Point

Batch ingestion starts from:

```
jobs/ingest_tickets.py
```

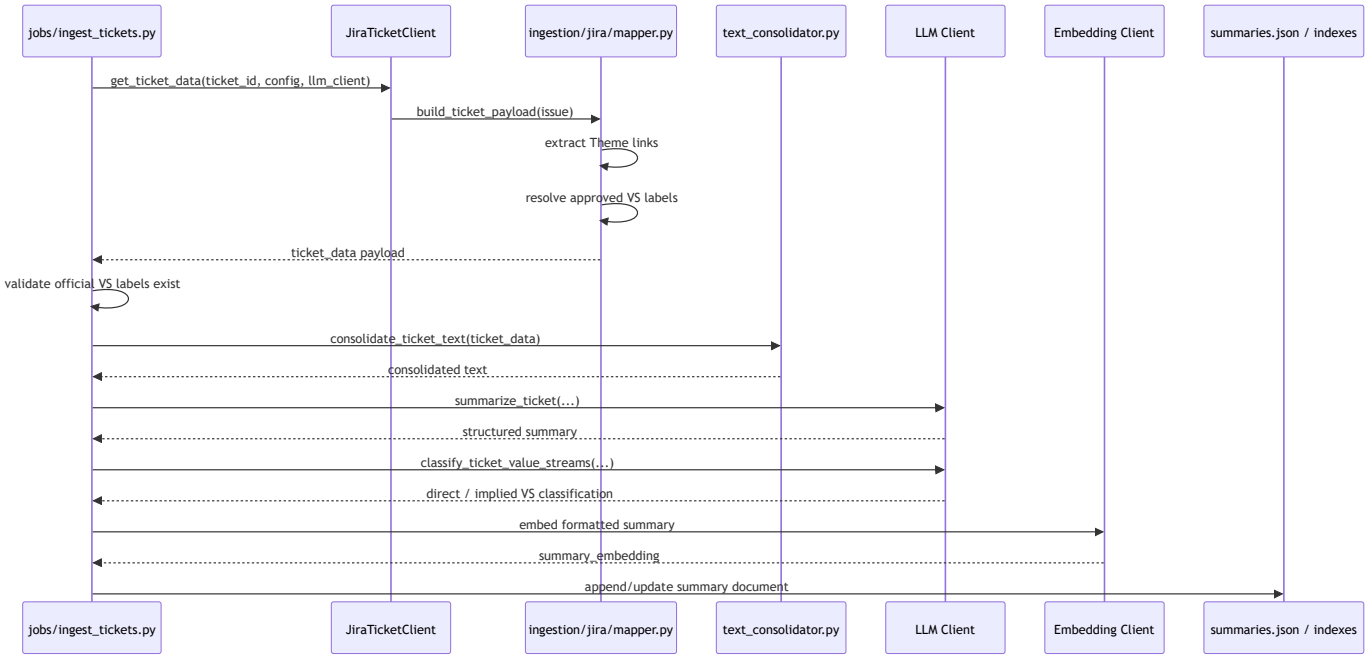
Example:

```
py -3 jobs/ingest_tickets.py IDMT-19761 --force --build-faiss
```

Common outputs:

```
ticket_data/summaries.json
ticket_data/_faiss/
ticket_data/ERROR_<ticket_id>.json
ticket_data/_errors.json
```

Runtime Flow



Stage 1: Fetch Jira Ticket

The job builds a Jira client through:

```
build_ticket_fetcher(...)
```

Then for each ticket:

```
ticket_data = await jira_client.get_ticket_data(
    ticket_id,
    config=cfg,
    llm_client=llm_client,
)
```

The fetched payload is not just raw Jira. It is transformed into the canonical ingestion payload.

Stage 2: Build Canonical Ticket Payload

File:

```
src/vs_app/ingestion/jira/mapper.py
```

Responsibility:

```
Raw Jira issue JSON → canonical ticket payload
```

It returns a dict containing:

```
key
fields
attachments
themes
value_stream_ids
value_stream_names
jira_group_ids
value_stream_statuses
linked_value_streams
value_stream_label_source
```

Important: this mapper should stay pure.

```
No REST calls.
No file downloads.
No embeddings.
No summary generation.
```

Stage 3: Official Value Stream Extraction

Source of truth

Official value-stream labels should come from Jira Theme links.

The preferred source is:

```
implemented-by / implements GROUP/THEME Jira issue links
```

The code extracts only linked Jira issues whose issue type contains:

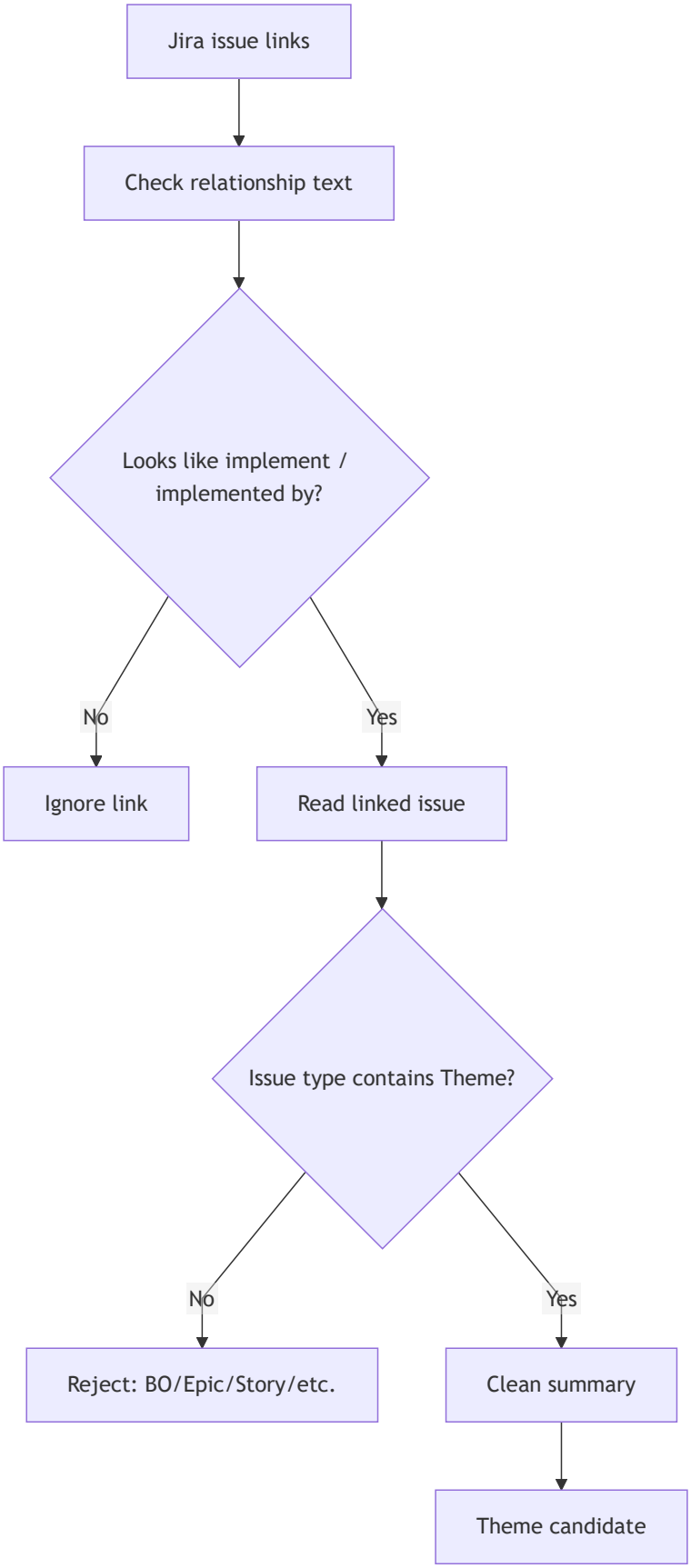
```
Theme
```

This prevents Business Objective / BO links from being treated as value streams.

Theme extraction flow

File:

```
src/vs_app/ingestion/jira/value_stream_labels/theme_extraction.py
```



Why Theme-only matters

Do not use this rule:

```
key starts with GROUP-
```

because Business Objectives may also use `GROUP-*`.

Correct rule:

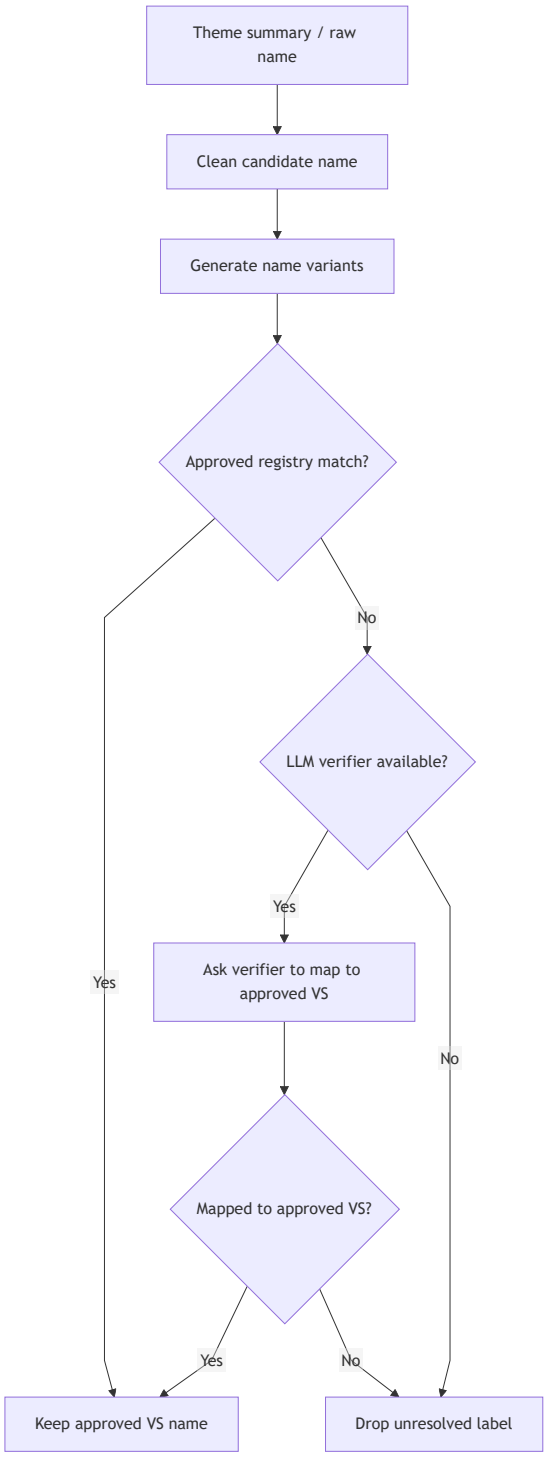
```
relationship looks implementation-related
AND linked Jira issue type contains Theme
AND final name resolves to approved value-stream registry
```

Stage 4: Canonical Value Stream Resolution

File:

```
src/vs_app/ingestion/jira/value_stream_labels/value_stream_mapping.py
```

Resolution order:



Important rule:

```
Never keep unresolved cleaned free-text as an official value stream.
```

If neither the registry nor the verifier maps the value to an approved value stream, drop it and log a warning.

Stage 5: Strict Label Requirement

The ingestion job refuses to index tickets without official value-stream labels.

In the job flow:

```
If value_stream_names and value_stream_ids are both empty
→ raise RuntimeError
→ write ERROR_<ticket_id>.json
→ do not write this ticket into summaries.json
```

Reason:

```
Historical RAG needs supervised historical examples.
Unlabeled tickets pollute the corpus.
```

Stage 6: Text Consolidation

File:

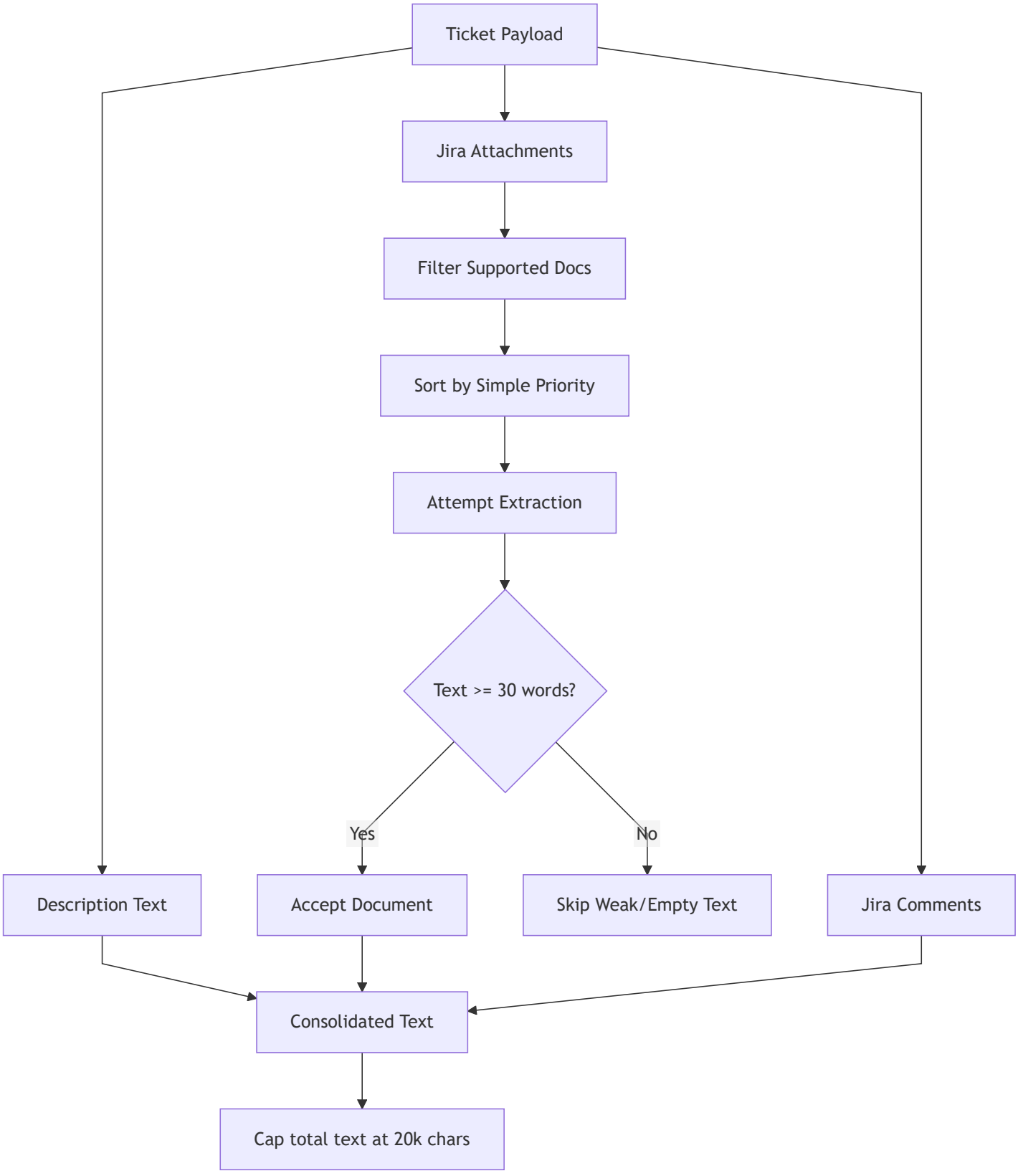
```
src/vs_app/ingestion/summary/text_consolidator.py
```

Text sources:

```
1. Jira description
2. Jira document attachments
3. Substantive Jira comments
```

No SharePoint.
No description-link synthetic attachments.
No heuristic idea-card route.

Consolidation diagram



Attachment Selection

Supported extensions:

```
pptx
ppt
pdf
docx
doc
```

Priority order:

```
1. Filename contains "idea" or "card"
2. Filename contains "business case", "proposal", "deck", or "initiative"
3. Extension priority:
   pptx → pdf → docx → ppt → doc
4. Smaller file size
5. Filename
```

Default limits:

```
max_documents = 4 accepted documents
max_prefetch_attachment_size = 60 MB
max_attachment_chars = 12,000 per accepted attachment
min_attachment_words = 30
```

Important behavior:

```
max_documents means accepted documents, not attempted documents.
If one document fails or is skipped, the pipeline continues to the next document.
```

This avoids losing useful attachments just because an earlier file failed extraction.

Text Limits

Current consolidation limits:

```
Description:          8,000 chars
Each attachment:     12,000 chars
Each comment:         1,500 chars
Total consolidated:  20,000 chars
Comments: up to 3 substantive comments
```

These limits protect the LLM prompt size while preserving the most useful evidence.

Stage 7: LLM Summary

File:

```
src/vs_app/ingestion/summary/pipeline.py
```

The summary pipeline requires an LLM client.

It refuses:

```
llm_client is None
skip_llm_summary=True
```

Reason:

```
No heuristic/fallback summaries should enter the historical RAG index.
```

The LLM creates a structured summary document from the consolidated text.

Stage 8: Direct / Implied Classification

After summarization, the pipeline asks the LLM to classify the official Jira value streams as:

```
direct
implied
```

Definitions:

```
direct:
  The ticket text clearly discusses the value stream.

implied:
  The value stream is likely impacted by the business pattern, but may not be directly name
d.
```

This classification is important for historical RAG because later, similar tickets can reveal implied value streams that a new idea card does not explicitly mention.

Stage 9: Embedding

The pipeline formats the structured summary and creates an embedding.

If embedding fails, ingestion raises an error.

Reason:

```
A ticket without embedding cannot reliably participate in historical retrieval.
```

Stage 10: Output Document

The final output is a `TicketSummaryDocument`.

Common fields:

```
ticket_id
title
summary
business_summary
technical_summary
value_stream_ids
value_stream_names
jira_group_ids
label_source
value_streams
direct_vs_names
implied_vs_names
summary_embedding
```

This document is written into:

```
ticket_data/summaries.json
```

and can be used to build:

```
ticket_data/_faiss/
Azure AI Search historical summary index
```

Terminal Progress

The batch job prints progress directly to terminal using `print(..., flush=True)`.

Expected flow:

```
[IDMT-19761] START ingest
[IDMT-19761] FETCHING Jira payload
[IDMT-19761] FETCHED Jira payload
[IDMT-19761] VS labels: 12 names
[IDMT-19761] SUMMARY/CLASSIFICATION started
[IDMT-19761] ATTACHMENTS found=5 supported_docs=2 max_documents=4
[IDMT-19761] ATTACHMENT attempting 1/2: Idea Card.pptx size=123456
[IDMT-19761] ATTACHMENT accepted 1/4: Idea Card.pptx words=850 chars=5200
[IDMT-19761] CLASSIFICATION direct=8 implied=4
[IDMT-19761] EMBEDDING complete dim=3072
[IDMT-19761] SAVED to ticket_data/summaries.json
[IDMT-19761] COMPLETE
```

Failure Behavior

Failure	Behavior
No LLM client	Fail
`--no-llm` used	Fail immediately
No official Theme VS labels	Fail ticket, do not index
Attachment unsupported	Skip attachment
Attachment too large and no pre-extracted text	Skip attachment
Attachment extraction error	Skip attachment
All attachments skipped	Continue with description/comments
LLM summary failure	Fail ticket
Direct/implied classification failure	Fail ticket
Embedding failure	Fail ticket

What Ingestion Does Not Do

Ingestion does **not**:

```
- decide final value streams for a new idea card
- run RAG candidate selection
- auto-select candidates
- use historical rescue logic
- accept SharePoint files
- use heuristic fallback summaries
- use Business Objective links as official VS labels
```

Ingestion only prepares reliable historical examples.

Architecture Summary

```
Jira ticket
→ Theme-only VS label extraction
→ Approved registry / verifier canonicalization
→ Strict labeled-ticket validation
→ Description + Jira docs + comments
→ LLM summary
→ LLM direct/implied classification
→ Embedding
→ Historical summary document
```

The result is a clean historical corpus that RAG can use safely.